

Exercice 1 :**Tableaux associatifs immuables avec des ABR**

Dans le fichier `abr.ml`, on définit le type `('a, 'b) abr`, correspondant aux arbres binaires de recherche étiquetés par des clés de type `'a`, avec une valeur de type `'b` associée à chaque clé, ainsi :

```
1 type ('a, 'b) abr = Vide | Noeud of ('a, 'b) abr * 'a * 'b * ('a, 'b) abr
```

On suppose que le type `'a` est totalement ordonné (et on utilise les opérateurs usuels pour les comparaisons) et immuable.

1. On remarque que la définition ne garantit pas que la propriété des ABR est vérifiée.

- (a) Écrire deux fonctions `mini : ('a, 'b) abr -> 'a * 'b` et `maxi : ('a, 'b) abr -> 'a * 'b` qui, étant donné `A` ABR non vide qui vérifie bien la définition vue en cours, calculent respectivement les étiqutes de nœuds de `A` de plus petite clé et de plus grande clé.
- (b) Écrire une fonction `est_abr : ('a, 'b) abr -> bool` qui vérifie qu'un arbre de type `abr` correspond bien à la définition d'un arbre binaire de recherche.
- (c) Montrer que si l'arbre est bien un ABR, votre fonction est de complexité au moins linéaire en n la taille de l'arbre¹. Écrire une autre fonction sur le même principe de complexité linéaire en n . Pourquoi ne peut-on pas faire mieux ?

Dans toutes les questions suivantes, on suppose de tout valeur de type `abr` passée en argument respecte la définition d'un ABR. Notamment, on n'appellera **pas** la fonction `est_abr` pour le vérifier.

2. Écrire les fonctions de spécifications suivantes (cf le fichier `abr.mli`) :

- (a) `mem : ('a, 'b) abr -> 'a -> bool` qui, étant donné un ABR `A` et une clé `k`, indique si `k` est une clé de `A`.
- (b) `find : ('a, 'b) abr -> 'a -> 'b` qui, étant donné un ABR `A` et une clé `k` présente dans `A`, renvoie la valeur associée à `k` dans `A`.
Si `k` n'est pas dans `A`, cette fonction déclenchera une erreur.
- (c) `find_opt : ('a, 'b) abr -> 'a -> 'b option` qui, étant donné un ABR `A` et une clé `k`, renvoie `Some` la valeur associée à `k` dans `A` si `k` est présente dans `A`, et `None` sinon.
- (d) `replace : ('a, 'b) abr -> 'a -> 'b -> ('a, 'b) abr` qui, étant donné un ABR `A`, une clé `k` et une valeur `v`, renvoie un ABR `A'` identique à `A` sauf que `k` y est associée à `v`.
- (e) `remove : ('a, 'b) abr -> 'a -> ('a, 'b) abr` qui, étant donné un ABR `A` et une clé `k`, renvoie un ABR `A'` identique à `A` sauf que la clé `k` n'y est pas.

3. Comparer l'efficacité de votre implémentation avec les tables de hachage. Pour ce faire, utilisez le fichier `test.ml`.

1. Prouver qu'elle est en fait linéaire et pas plus coûteuse peut être difficile.

Exercice 2 :**Files de priorités mutables**

Dans ce TP, on va implémenter les files de priorités en utilisant un tas binaire. Pour cela, on remarque qu'un arbre complet A de hauteur h (et donc de taille $n \in \llbracket 2^h; 2^{h+1} - 1 \rrbracket$) peut aisément se représenter par la donnée de n et d'un tableau t de taille (au moins) 2^{h+1} tel que :

- $t[0]$ est inutilisé
- Si A n'est pas vide, $t[1]$ est sa racine.
- Pour $2 \leq i \leq n$, $t\left[\left\lfloor \frac{i}{2} \right\rfloor\right]$ est le parent de $t[i]$.
- Pour $i \geq 1$, les éventuels enfants de $t[i]$ sont $t[2i]$ et $t[2i + 1]$.

Pour ne pas refaire le travail déjà fait pour les tableaux redimensionnables, un fichier `tas.c` vous est fourni avec la définition de la structure et des fonctions `tas_create`, `tas_delete`, `tas_is_empty` et `tas_incr_h` déjà écrites.

Ici, on suppose que chaque élément du tas a une priorité et une valeur toutes deux entières, même si pour l'instant on n'utilise que la priorité.

- (a) Écrire une fonction `void tamisage_haut(tas* t, couple x, int i)` qui prend en argument un couple $x = (p, v)$ et un entier $i \in \llbracket 1; n \rrbracket$ tel que p la priorité de x minore celle de tous les descendants de la case d'indice i dans t .
On suppose de plus que la case d'indice i est libre, c'est à dire qu'elle est nouvellement créé ou que le couple qui y était a été déplacé.
Cette fonction tente de placer x à l'indice i . Si ce n'est pas possible car x est plus petit que le parent de i , alors on déplace le parent en i , puis on réessaie à la place du parent.
- (b) En utilisant la fonction précédente, écrire une fonction `void tas_enfile(tas* t, couple x)` qui ajoute un couple x à un tas binaire. Attention au cas où le tas binaire atteint sa capacité.
- Écrire une fonction `couple tas_min(tas* t)` qui renvoie le couple de priorité minimale d'un tas binaire non vide.
- Pour retirer la racine d'une file de priorité, on remplace son contenu par celui du dernier nœud du tas (que l'on supprime), puis on filtre vers le bas plutôt que vers le haut la nouvelle priorité de la racine (qui peut ne pas être à sa place).
 - Écrire une fonction `void tamisage_bas(tas* t, couple x, int i)` qui prend en argument une valeur $x = (p, v)$ et un entier $i \in \llbracket 1; n \rrbracket$ tel que p majore tous les ascendants de la case d'indice i dans t .
On suppose de plus que la case d'indice i est libre, c'est à dire qu'elle est nouvellement créé ou que le couple qui y était a été déplacé.
Cette fonction tente de placer x à l'indice i . Si ce n'est pas possible car p est plus grande que la priorité d'un des enfants de i , alors on déplace l'enfant avec la plus petite priorité en i , puis on réessaie à la place de cet enfant.
 - En utilisant la fonction précédente, écrire une fonction `couple tas_defile(tas* t)` qui retire le minimum d'un tas binaire non vide et le renvoie. Attention au cas où le tas binaire devient vide.
- Écrire une fonction `tas* tas_of_array(int* a, int n)` qui crée un tas binaire contenant les éléments d'un tableau (on mettra une valeur arbitraire, par exemple 0, pour le deuxième élément du couple).
Remarque : Comme on le verra en cours, il est possible d'écrire cette fonction avec une complexité $\Theta(n)$ plutôt que $\Theta(n \log(n))$
- En déduire une fonction `void tri_par_tas(int* a, int n)` qui trie un tableau d'entiers en temps quasi-linéaire en utilisant un tas binaire.